Gestion des Erreurs

Objectifs

1. Définir et exposer des **erreurs sentinelles** (`Err...`) conformes aux conventions de la bibliothèque standard Go.
2. Créer des **structures d'erreurs personnalisées** (`...Error`) implémentant l'interface native `error` et la méthode `Unwrap() error`.
3. Enrichir les erreurs à chaque traversée de couche logicielle grâce au verbe **`%w`** de `fmt.Errorf` tout en respectant la règle *"Handle errors once"*.
4. Dérouler et inspecter une chaîne d'erreurs complexe à l'aide de **`errors.Is`** et **`errors.As`**.

Contexte métier : Pipeline de virement interbancaire

Vous développez le moteur de virement SEPA d'une néobanque. Une demande de virement traverse **3 couches logicielles** successives :

1. **Couche Bas Niveau (`verifierConformiteEtSolde`) :** Vérifie l'état du compte en base et les règles anti-fraude.
2. **Couche Service Métier (`executerVirement`) :** Appelle la couche bas niveau et enrichit l'erreur éventuelle avec l'identifiant du virement et l'IBAN cible.
3. **Couche Contrôleur API (`traiterRequeteHTTP` dans `main`) :** Appelle le service métier, inspecte la cause racine de l'échec et traduit l'erreur technique en code de statut HTTP approprié (`400`, `403`, `422` ou `503`) sans jamais dupliquer les logs.

Exercice 1 : Erreurs sentinelles et Erreurs typées personnalisées

1. **Définition des Erreurs Sentinelles :** En haut de votre fichier, déclarez avec `errors.New` trois erreurs sentinelles respectant les conventions Go (préfixe `Err`, pas de majuscule au début du message, pas de point final) :
    - `ErrMontantInvalide` (`"le montant du virement doit être strictement positif"`)
    - `ErrCompteGele` (`"opération refusée : le compte est sous blocage administratif"`)

- `ErrReseauIndisponible("délai d'attente dépassé vers la chambre de compensation")`

2. **Création d'une Structure d'Erreur Personnalisée (`SoldeInsuffisantError`) :** Lorsqu'un compte n'a pas assez de fonds, une simple erreur sentinelle ne suffit pas : le front-end a besoin de connaître le solde actuel et le montant exact qu'il manque pour proposer un rechargement instantané.
    - Créez une structure `SoldeInsuffisantError` contenant :
        - `CompteID` (`string`)
        - `SoldeActuel` (`float64`)
        - `MontantRequis` (`float64`)
    - Implémentez l'interface `error` sur `*SoldeInsuffisantError` (méthode `Error() string`) pour retourner un message formaté précis (ex: `"solde insuffisant sur le compte FR-99 : disponible 150.00€, requis 500.00€"`).
    - Ajoutez une méthode utilitaire `MontantManquant() float64` sur `*SoldeInsuffisantError` qui retourne `e.MontantRequis - e.SoldeActuel`.
3. **Création d'une Structure d'Erreur avec cause chaînée (`PasserelleError`) :**
    - Créez une structure `PasserelleError` contenant `NomPasserelle string`, `CodeInterne int` et `Cause error` (l'erreur d'origine).
    - Implémentez `Error() string` ainsi que la méthode **`Unwrap() error`** (qui retourne `e.Cause`) afin que `errors.Is` et `errors.As` puissent traverser cette structure automatiquement !

Exercice 2 : Propagation et *Error Wrapping* (`%w`) à travers les couches

1. **Couche 1 — Validation et accès aux données :** Écrivez la fonction `verifierConformiteEtSolde(compteID string, solde float64, montant float64) error` qui applique les règles suivantes :
    - Si `montant <= 0`, retourne `ErrMontantInvalide`.
    - Si `compteID == "COMPTE-GELE"`, retourne `ErrCompteGele`.

- Si `compteID == "COMPTE-TIMEOUT"`, retourne un pointeur `&PasserelleError{NomPasserelle: "SWIFT-EU", CodeInterne: 504, Cause: ErrReseauIndisponible}`.
- Si `montant > solde`, retourne un pointeur `&SoldeInsuffisantError{CompteID: compteID, SoldeActuel: solde, MontantRequis: montant}`.
- Sinon, retourne `nil`.

2. **Couche 2 — Service Métier (Enrichissement sans logger) :** Écrivez la fonction `executerVirement(idTransaction string, compteID string, solde float64, montant float64) error` :
   - Elle appelle `verifierConformiteEtSolde(compteID, solde, montant)`.
   - Si une erreur survient, **conformément à la règle *"Handle errors once"*, n'affichez aucun log ici**, mais retournez une erreur enrichie avec `fmt.Errorf` et le verbe **`%w`** :

```
None
return fmt.Errorf("échec transaction [%s] pour le compte %s: %w", idTransaction, compteID, err)
```

Exercice 3 : Inspection profonde avec `errors.Is` et `errors.As`

Vous êtes maintenant au sommet de la pile d'appels (le contrôleur HTTP simulé). C'est le **seul endroit** où l'erreur doit être journalisée et traduite en décision métier.

1. **Démonstration du piège de l'opérateur `==` :**
   - Appelez `err := executerVirement("TX-001", "COMPTE-GELE", 1000, 200)`.
   - Testez d'abord `if err == ErrCompteGele` : vérifiez que cette condition est **fausse** bien que la cause racine soit `ErrCompteGele`. Expliquez pourquoi.
2. **Aiguillage intelligent avec `errors.Is` et `errors.As` :** Écrivez une fonction `traiterRequeteVirement(idTx, compte string, solde, montant float64)` qui appelle `executerVirement` et inspecte l'erreur retournée :

- Si `err == nil` : affichez `[HTTP 200 OK] Virement <idTx> validé`.
- Si `errors.Is(err, ErrMontantInvalide)` : affichez `[HTTP 400 Bad Request]` + le message complet de `err`.
- Si `errors.Is(err, ErrCompteGele)` : affichez `[HTTP 403 Forbidden] Alerte conformité` + le message complet de `err`.
- Si `errors.Is(err, ErrReseauIndisponible)` (pourtant enfermée dans `PasserelleError` elle-même enfermée dans `fmt.Errorf("%w")` !) : affichez `[HTTP 503 Service Unavailable] Réessai automatique programmé`. Bonus : extrayez également la structure `*PasserelleError` avec `errors.As` pour afficher son `CodeInterne`.
- Si `errors.As` détecte une erreur de type `*SoldeInsuffisantError` : affichez `[HTTP 422 Unprocessable Entity]` et appelez directement sa méthode `MontantManquant()` pour afficher : `"Action requise : veuillez créditer au moins X.XX € sur votre compte"`.

3. Dans `main()`, exécutez une batterie de 5 appels à `traiterRequeteVirement` couvrant tous les scénarios (succès, montant négatif, compte gelé, panne réseau SWIFT, et solde insuffisant).